stacked

Stacked cubes and dices

v0.2.0 Akilon <https://github.com/Akilon27> MIT

About

Cube stacks were introduced into the French middle-school curriculum in 2025, which is what motivated the creation of this package.

I was inspired by `tikz3d-fr` (author: Cédric Pierquet) and by part of `ProfCollege` for LaTeX (author: Christophe Poulain), but this implementation is not a copy of either. I used AI (Gemini and Claude) for the dice and the random part of the `pile` function; the rest is my own work, except for the `dice-face` function, which was created by eric1102 (on Discord) and is included here with his permission.

Version 2 adds many functions for representing solids (polyhedra, cones, cylinders, and spheres), their cross-sections by a plane (defined either by three points or by the four coefficients of its Cartesian equation), and their nets. Helper functions are provided to automatically compute vertex coordinates for some common cases (pyramids, prisms, and several regular polyhedra: tetrahedron, cube, octahedron, dodecahedron, and icosahedron). In addition, the globe function can be used to explain geographical and spherical coordinates.

v1 functions: cube stacks and dice

- cube()
- pile()
- building()
- above-view()
- right-view()
- front-view()
- below-view()
- left-view()
- back-view()
- maincoords()
- dice()
- throws()
- dice-face()

cube

CeTZ command for a cube.

Parameters

```
cube(
  a: array,
  c: auto color array,
  border-stroke: boolean color,
  thickness: int float ratio,
  light: float ratio array,
  side: boolean
) -> function
```

a `array`

Position

c `auto` or `color` or `array`

auto → (red on the right, green on top, yellow on the left), otherwise one or 3 colors

Default: `auto`

border-stroke `boolean` or `color`

Lines in black or in the cube color, default false

Default: `false`

thickness `int` or `float` or `ratio`

Stroke thickness (from 1pt), default 100%

Default: `100%`

light `float` or `ratio` or `array`

How much to lighten when using a single color, 20% by default

Default: `20%`

side `boolean`

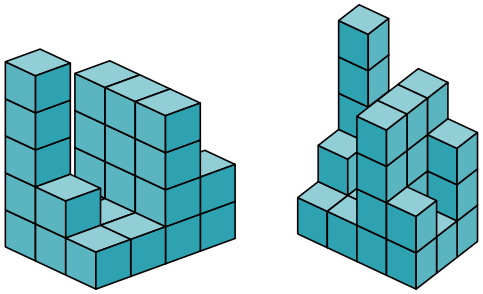
Internal parameter for the side view with `pile`

Default: `false`

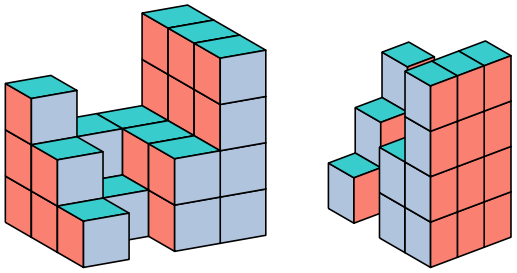
pile

Stack of small cubes on a grid, given as a list from back to front `3251`, then from left to right (separated by commas). A color can be specified at the beginning or the end. With the default colors using none: front → yellow, top → green, right → red. Alternatively, a list of 3 colors can be provided with colors.

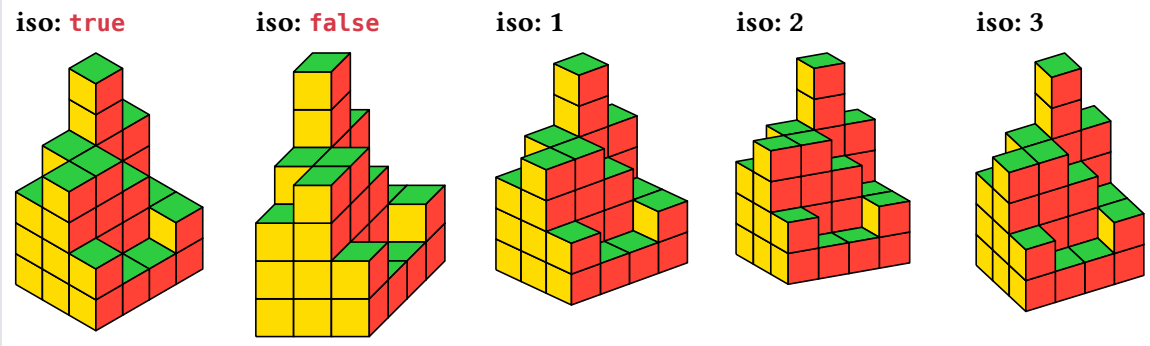
```
#pile(3415,1412,2411,eastern,side:true,
iso:1,draw-scale:.6,border-stroke:true)
```



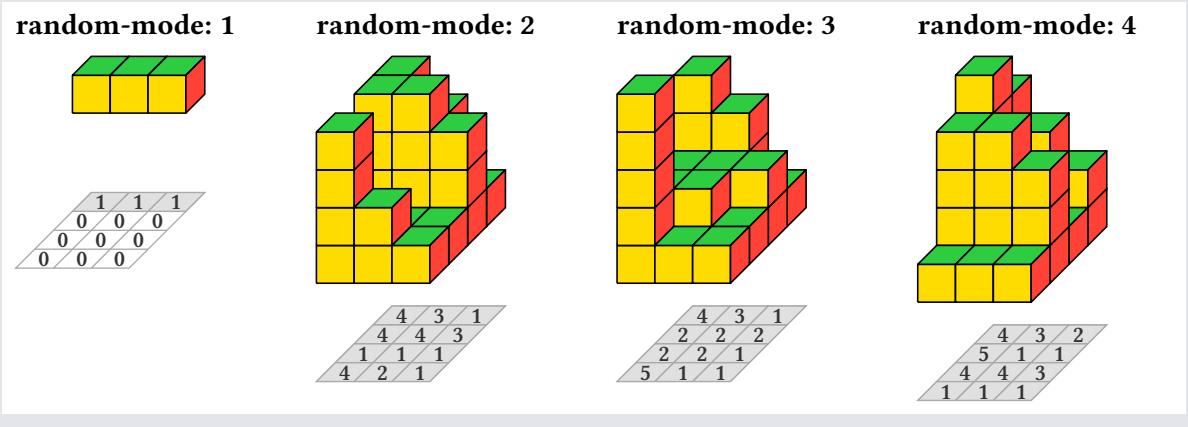
```
#pile(4223,4212,4201,colors:
(rgb("#fa8072"),rgb("#b0c4de"),teal),
side: true,iso: 2,draw-scale: .75)
```



```
#grid(columns: 5*(1fr,), column-gutter: 1.5em,
 ..for i in (true,false,1,2,3) {([=== iso: #i
   #pile("4643",2344,2112, iso: i)
 ],)}
 )
```



```
#grid(columns: 5*(3.2cm,), column-gutter: 2em,
 ..for i in (1,2,3,4) {([=== random-mode: #i
   #pile(random:true, random-mode: i,shadow:true,seed:2*i,plan:true)
 ],)}
 )
```



`z:0` for front view, `y:0, z:(0,-1)` for top view, `x:0, z:(-1,0)` for right view

Parameters

```
pile(
  ..a: int str array arguments,
  random: boolean,
  width: int,
  depth: int,
  height: int,
  seed: int,
  plan: boolean,
  labels: boolean,
  x: array,
  y: array,
  z: array,
  thickness: int float ratio,
  border-stroke: boolean color,
  side: boolean,
  light: float ratio array,
  draw-scale: float ratio,
  x-spread: float int,
  y-spread: float int,
  z-spread: float int,
  colors: array,
  grids: boolean,
  solution: boolean,
  below-separation: auto int float,
  left-separation: auto int float,
  back-separation: auto int float,
  iso: boolean int,
  space: length relative length fraction,
  holes: boolean,
  random-mode: auto int,
  views: array,
  views-names: array,
  axes: boolean,
  opacity: ratio,
  shadow: boolean,
  disposition: str,
  box-args: dictionary
) -> content
```

..a `int` or `str` or `array` or `arguments`

List of numbers corresponding to the heights back-to-front, left-to-right

random `boolean`

Enables a random mode: automatically generates the stack (instead of positional arguments), with equal or decreasing heights from row to row

Default: `false`

width `int`

Width (number of slices along X) in random mode

Default: `3`

depth `int`

Depth (number of digits per slice along Z) in random mode

Default: `4`

height `int`

Maximum height (along Y) in random mode

Default: `5`

seed `int`

Seed of the pseudo-random generator for reproducible results: same seed → same stack; change it to obtain another one

Default: `1`

plan `boolean`

Display the level plan (numbered top view)

Default: `false`

labels `boolean`

Display the view labels on the grids

Default: `false`

x `array`

Direction of the first coordinate, default (1,0)

Default: `(1,0)`

y `array`

Direction of the second coordinate, default (0,1)

Default: `(0,1)`

z `array`

Direction of the third coordinate, default (-0.5,-0.5)

Default: `(-.5,-.5)`

thickness `int` or `float` or `ratio`

Stroke multiplier (from 1pt), default 60%, i.e. 0.6pt

Default: `60%`

border-stroke `boolean` or `color`

Lines in black, default false

Default: `false`

side `boolean`

Add a second view of the assembly from the “right”

Default: `false`

light `float` or `ratio` or `array`

How much to lighten when using a single color

Default: `20%`

draw-scale `float` or `ratio`

Changes the edge size of a base cube

Default: `.5`

x-spread `float` or `int`

Allows the generated stack to be “exploded” to the right to make visualization by “slices” easier

Default: `0`

y-spread `float` or `int`

Allows the generated stack to be “exploded” upward to make visualization by “layers” easier

Default: `0`

z-spread `float` or `int`

Allows the generated stack to be “exploded” forward to make visualization by “slices” easier

Default: `0`

colors `array`

Provide a list of 3 colors for the three faces (left, right, top)

Default: `()`

grids `boolean`

Display three grids allowing the student to directly draw the front, top, and left views

Default: `false`

solution `boolean`

When set to true, display the associated solid’s front, top, and left views on the grids

Default: `false`

below-separation `auto` or `int` or `float`

Separation of the bottom grid (auto = automatically calculated)

Default: `auto`

left-separation `auto` or `int` or `float`

Separation of the left grid (auto = automatically calculated)

Default: `auto`

back-separation `auto` or `int` or `float`

Separation of the back grid (auto = automatically calculated)

Default: `auto`

iso `boolean` or `int`

Allows different perspectives:
- true = isometric representation,
- 1 = tikz3d-fr view,
- 2 = first ProfCollege view,
- 3 = second ProfCollege view,
- otherwise = default CeTZ view

Default: `false`

space `length` or `relative length` or `fraction`

Spacing from the second view

Default: `2em`

holes `boolean`

Allows “holes” (zero height) in the random stack starting from the second stack; the first stack always remains “full” (height >= 1 everywhere)

Default: `false`

random-mode `auto` or `int`

Specific random mode:
- 1 (double-min LCG by Gemini),
- 2 and 4 (decreasing LCG inspired by ProfCollege by Claude),
- 3 (suiji package)

Default: `auto`

views `array`

List of views to display among (“top”, “front”, “right”)

Default: `("dessus", "face", "droite")`

views-names `array`

Names of the displayed views among (“top”, “front”, “right”)

Default: `("Vue de dessus", "Vue de face", "Vue de droite")`

axes `boolean`

Display axis markers (coordinates) on the ground

Default: `false`

opacity `ratio`

Opacity of the cube faces (100% = opaque, 50% = semi-transparent)

Default: `100%`

shadow `boolean`

Draw the footprint on the ground beneath the stack

Default: `false`

disposition `str`

Layout of the 2D grids: “3d” (projected around) or “cote-a-cote” (next to the 3D view)

Default: `"3d"`

box-args `dictionary`

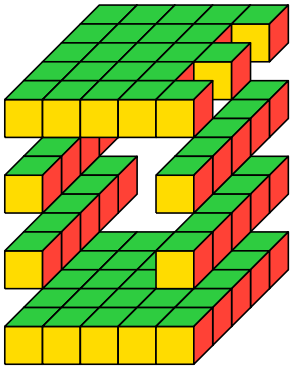
Arguments to pass to the box(es)

Default: `()`

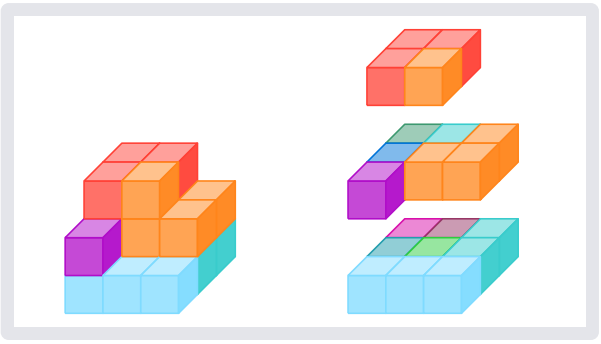
building

Construct of small cubes with mixed colors: each number corresponds to the position in the color list (16 max: 1 to 9 and a to g + t for 3 colors). “0” and “o” represent a hole, “x” uses the first color. Cubes are given as a list of lists: from front to back using a number/string such as `"5432a"`, then layer by layer `btt`, with lists ordered from left to right.

```
#building((
("ttttt","ttttt","ttttt","ttttt"),
// Leftmost slice, from bottom to top
("ttttt","oooooo","oooooo","ttttt"),
("ttttt","oooooo","oooooo","ttttt"),
("ttttt","oooooo","oooooo","ttttt"),
("ttttt","ttttt","ttttt","totot")
// Rightmost slice, same arrangement
), y-spread:1)
```



```
#building(((578,"21b",44),
("59a",63,64), (533,66,0),))
#h(3em)
#building(((578,"21b",44),
("59a",63,64), (533,66,0),),
y-spread:1.5)
```



`z:0` for front view, `y:0, z:(0,-1)` for top view, `x:0, z:(-1,0)` for right view

Parameters

```
building(
  ..a: array arguments,
  color-list: array,
  x: array,
  y: array,
  z: array,
  thickness: int float ratio,
  border-stroke: boolean color,
  _rev,
  _dessous,
  _back,
  light: float ratio array,
  draw-scale: float ratio,
  iso: boolean int,
  text: str content,
  x-spread: float int,
  y-spread: float int,
  z-spread: float int,
  box-args: dictionary
) -> content
```

..a `array` or `arguments`

List of list of numbers corresponding to the colors back-to-front, bottom to top, left-to-right

color-list `array`

List of up to 16 colors

Default: `typst16`

x `array`

Direction of the first coordinate, default (1,0)

Default: `(1,0)`

y `array`

Direction of the second coordinate, default (0,1)

Default: `(0,1)`

z `array`

Direction of the third coordinate, default (-0.5,-0.5)

Default: `(-.5,-.5)`

thickness `int` or `float` or `ratio`

Stroke multiplier (from 1pt), default 60%, i.e. 0.6pt

Default: `60%`

border-stroke `boolean` or `color`

Lines in black, default false

Default: `false`

light `float` or `ratio` or `array`

light:20% -> how much to lighten when using a single color

Default: `20%`

draw-scale `float` or `ratio`

Changes the edge size of a base cube

Default: `.5`

iso `boolean` or `int`

Allows different perspectives:
- true = isometric representation,
- 1 = tikz3d-fr view,
- 2 = first ProfCollege view,
- 3 = second ProfCollege view,
- otherwise = default CeTZ view

Default: `false`

text `str` or `content`

Text to place below the stack (for the views)

Default: `none`

x-spread `float` or `int`

Allows the generated stack to be “exploded” to the right to make visualization by “slices” easier

Default: `0`

y-spread `float` or `int`

Allows the generated stack to be “exploded” upward to make visualization by “layers” easier

Default: `0`

z-spread `float` or `int`

Allows the generated stack to be “exploded” forward to make visualization by “slices” easier

Default: `0`

box-args `dictionary`

Arguments to pass to the box

Default: `()`

above-view

Top view for a building. Change text to “Top view” or “”

Parameters

```
above-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments for building

text `str` or `content`

View name

Default: `"Vue du dessus"`

right-view
Right view for a building. Change text to “Right view”

Parameters

```
right-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments for building

text `str` or `content`

View name

Default: `"Vue de droite"`

front-view
Front view for a building. Change text to “Front view”

Parameters

```
front-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments for building

text `str` or `content`

View name

Default: `"Vue de face"`

below-view
Bottom view for a building. Change text to “Bottom view”

Parameters

```
below-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments for building

text `str` or `content`

View name

Default: `"Vue du dessous"`

left-view
Left view for a building. Change text to “Left view”

Parameters

```
left-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments for building

text `str` or `content`

View name

Default: `"Vue de gauche"`

back-view
Back view for a building. Change text to “Back view”

Parameters

```
back-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments for building

text `str` or `content`

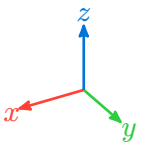
View name

Default: `"Vue arrière"`

maincoords

Function that sets the 3D viewing angle for a cetz.canvas using polar (θ) and azimuthal (φ) angles. You have to pass it directly to the canvas, without forgetting to spread.

```
#canvas(..maincoords(theta:60, phi:150), {
   import draw:*
   repere
 })
```



Parameters

```
maincoords(
  theta: int angle,
  phi: int angle
) -> dictionary
```

theta `int` or `angle`

Polar angle θ

Default: `70`

phi `int` or `angle`

Azimuthal angle φ

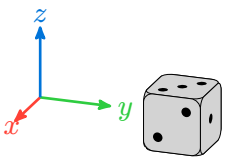
Default: `110`

dice

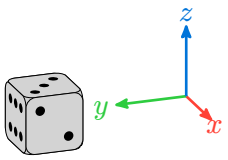
Function used to draw a 3D die.

Warning, using dice in a canvas overwrites the transformation matrix!

```
#canvas({
   import draw:*
   dice(standalone:false, origin: (0,2),
   front:2, top:3, s:.8, vue: "D")
   repere
 })
```



```
#canvas({
   import draw:*
   dice(standalone: false, origin: (0,2),
   front: 2, top:3, s:.8, vue: "G")
   repere
 })
```



Parameters

```
dice(
  standalone: boolean,
  front: int,
  top: int,
  s: int float ratio,
  theta: int float,
  phi: int float,
  vue: str,
  color: color,
  dot-color: color,
  origin: array,
  box-args: dictionary,
  opacity: ratio
) -> content function
```

standalone `boolean`

true if alone, false if already inside a canvas

Default: `true`

front `int`

Number of spots on the front face

Default: `1`

top `int`

Number of spots on the top face

Default: `2`

s `int` or `float` or `ratio`

Drawing scale

Default: `1.0`

theta `int` or `float`

Theta angle of the spherical coordinates

Default: `70`

phi `int` or `float`

Phi angle of the spherical coordinates

Default: `110`

vue `str`

“D” for right view or “G” for left view

Default: `"D"`

color `color`

Die color, light gray by default

Default: `rgb("d3d3d3")`

dot-color `color`

Spot color

Default: `black`

origin `array`

Position of the die’s origin

Default: `(0,0,0)`

box-args `dictionary`

Additional arguments for the box

Default: `()`

opacity `ratio`

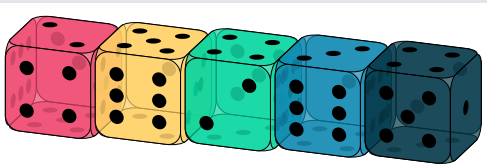
Opacité des dés

Default: `100%`

throws

Random or non-random roll of n dice.

```
#throws(5, yams: true, colors:
(rgb("ef476f"), rgb("ffd166"),
rgb("06d6a0"), rgb("118ab2"),
rgb("073b4c")), seed: 999, s: 1.2,
yams-h: -.15,  opacity: 70%,)
```



Parameters

```
throws(
  n: int,
  list: none array,
  seed: int,
  yams: boolean,
  yams-h: int float,
  espace-h: length,
  s: int float ratio,
  colors: array,
  dot-color: color,
  theta: int float,
  phi: int float,
  vue: str,
  box-args: dictionary,
  opacity: ratio
) -> content
```

n `int`

Number of dice

list `none` or `array`

List of rolls: none by default, otherwise a list of pairs (front face, top face)

Default: `none`

seed `int`

Pseudo-random seed

Default: `42`

yams `boolean`

Dice displayed “touching” each other

Default: `false`

yams-h `int` or `float`

Horizontal offset of the dice in the row when yams is enabled

Default: `.1`

espace-h `length`

Horizontal spacing between dice

Default: `5mm`

s `int` or `float` or `ratio`

Scale factor

Default: `1.0`

colors `array`

List of die colors

Default: `(rgb("d3d3d3"),)`

dot-color `color`

Spot color

Default: `black`

theta `int` or `float`

Theta angle of the spherical coordinates

Default: `70`

phi `int` or `float`

Phi angle of the spherical coordinates

Default: `110`

vue `str`

“D” for right view or “G” for left view

Default: `"D"`

box-args `dictionary`

Additional arguments for the box

Default: `()`

opacity `ratio`

Opacité des dés

Default: `100%`

dice-face

From eric1102 on Discord: draw one side of a die with the possibility to change the fill color, spot fill, and stroke

Parameters

```
dice-face(
  num: int,
  fill: none color,
  spot-fill: auto color,
  stroke: auto stroke
) -> content
```

num `int`

Int from 1 to 6

fill `none` or `color`

Color of the dice

Default: `none`

spot-fill `auto` or `color`

Black for a light-colored fill, white for a dark-colored fill, or a chosen color

Default: `auto`

stroke `auto` or `stroke`

Stroke of the dice, if a fill is provided and stroke:auto, same as the fill

Default: `auto`

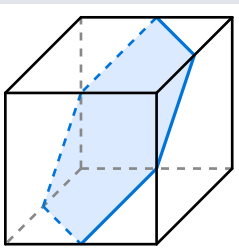
v2 functions for 3D solids (and nets and cross-sections)

- polysection()
- point-ratio()
- cylsection()
- conesection()
- doubleconesection()
- spheresection()
- globe-view()
- globe()
- polyunfold()
- cylunfold()
- coneunfold()
- regpolygon()
- regpyramid()
- regprism()
- regantiprism()
- regbipyramid()
- regpolyhedron()

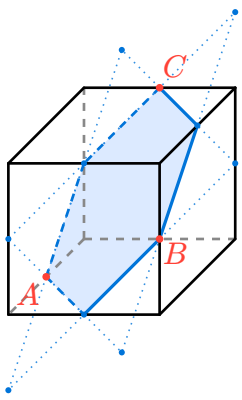
polysection

Draws a convex polyhedron and its cross-section by a plane in oblique projection (CeTZ). The polyhedron is defined by its list of vertices: faces are inferred automatically (convex hull). Specify the plane using three points or the four coefficients of its Cartesian equation. Anchors: “0”, “1”, … for vertices (in `vertices` order), followed by the intersection points of extensions (see `show-extensions` and `show-all`); `help: true` numbers them.

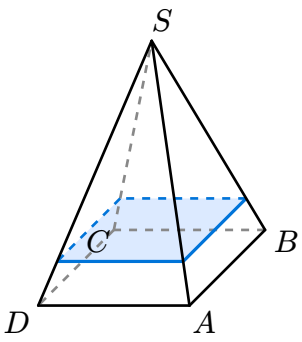
```
// Usage examples:
#polysection((-1, -1, 0), (1, 0, 1), (0, 1, -1))
```



```
// Plane defined by three named points, with extensions of the cross-section edges
#polysection((-1, -1, 0), (1, 0, 1), (0, 1, -1), plane-names: ($A$, $B$, $C$), show-extensions: true)
```



```
// Regular pyramid (vertices computed
by `regpyramid`) with vertex labels
#polysection(0, 1, 0, -0.5, vertices:
regpyramid(4, side: 2, height: 3),
  vertices-names: ($A$, $B$, $C$, $D$,
  $S$))
```



Parameters

```
polysection(
  vertices: array,
  ..plane-args: array dictionary,
  vertices-names: none array,
  plane-names: none array,
  show-extensions: bool,
  show-all: bool,
  x: array,
  y: array,
  z: array,
  eps: float,
  more: cetz commands,
  section-border: color,
  section-fill: color,
  help: bool,
  hidden-style: str array dictionary,
  extensions-style: str array dictionary
) -> content
```

vertices `array`

Vertices of the convex polyhedron, as points (x, y, z); defaults to the cube $[-1, 1]^3$. The `solidvertices` functions (`regpyramid`, `regprism`, `regpolyhedron`, ...) provide these

Default: (
```
    (-1, -1, -1), (1, -1, -1), (1, 1, -1), (-1, 1, -1),
    (-1, -1,  1), (1, -1,  1), (1, 1,  1), (-1, 1,  1)
  )
```

..plane-args `array` or `dictionary`

Cutting plane: three points (A, B, C), or four coefficients a, b, c, d of the equation ax + by + cz + d = 0, or a single argument (dictionary (a:, b:, c:, d:), array of 4 numbers, or array of 3 points)

vertices-names `none` or `array`

Vertex names, in `vertices` order

Default: `none`

plane-names `none` or `array`

Names of the three points defining the plane, shown in red (ignored if the plane is specified by an equation)

Default: `none`

show-extensions `bool`

Computes and displays extensions of the cross-section edges and their intersection points

Default: `false`

show-all `bool`

Also displays intersections between the lines containing the cross-section edges and the lines containing the solid edges

Default: `false`

x `array`

Screen image of the x-axis unit vector (oblique projection)

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (oblique projection)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) : by default (-0.5, −0.5), the z-axis points toward the viewer

Default: `(-0.5, -0.5)`

eps `float`

Numerical tolerance (distances)

Default: `1e-6`

more `cetz commands`

Additional CeTZ commands drawn on the same canvas (they can use the anchors)

Default: `()`

section-border `color`

Cross-section outline color

Default: `blue`

section-fill `color`

Cross-section fill (usually semi-transparent)

Default: `rgb(0, 102, 255, 35)`

help `bool`

Displays anchor numbers in red (vertices, then intersection points)

Default: `false`

hidden-style `str` or `array` or `dictionary`

Hidden-line style: the value of `dash` for a Typst stroke (“dashed”, “dotted”, …)

Default: `"dashed"`

extensions-style `str` or `array` or `dictionary`

Extension guide-line style

Default: `"dotted"`

point-ratio

Point located at a fraction of segment [A C], measured from A (in 2D or 3D). Useful for placing points in `more`, for example the midpoint of an edge or cross-section segment. `ratio` is a number between 0 and 1 or a percentage: `point-ratio((0, 0, 0), 25%, (4, 0, 0))` returns `(1, 0, 0)`.

Parameters

```
point-ratio(
  A: array,
  ratio: int float ratio,
  C: array
) -> array
```

A `array`

Starting point (x, y) or (x, y, z)

ratio `int` or `float` or `ratio`

Fraction of segment [A C]: number (0.25) or percentage (25%)

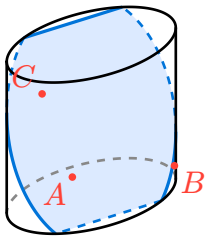
C `array`

End point, with the same dimension as A

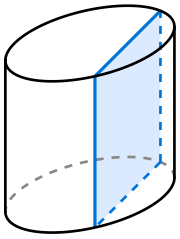
cylsection

Draws a cylinder and its cross-section by a plane in oblique projection (CeTZ). The cylinder is defined by its radius, height, base center, and axis direction. Specify the plane as for `polysection` (three points or four coefficients). Special cases are handled: a plane parallel to the axis (rectangle), perpendicular to the axis (circle); the ellipse can show its foci. Anchors: “c0”, “c1” (base centers), “i0”, “i1”, … (endpoints of the cross-section chords on the bases), “sc” (center of the cross-section when it is a complete ellipse), “f0”, “f1” (foyers).

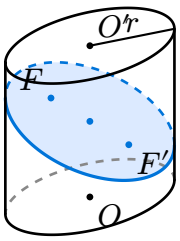
```
// Usage examples:
#cylsection((0, 0.5, 0.5), (1, 0.3,
-0.2), (-0.6, 1.4, 0.1), plane-names:
($A$, $B$, $C$))
```




```
// Plane parallel to the axis
(rectangle)
#cylsection(1, 0, 0, -0.5, radius: 1,
height: 2)
```



```
// Complete ellipse with a radius,
centers, and foci
#cylsection(0.6, 1, 0, -1, show-radius:
true, show-centers: true, show-foci:
true,
  point-labels: (c0: $O$, c1: $O'$, f0:
  $F$, f1: $F'$))
```



Parameters

```
cylsection(
  ..plane-args: array dictionary,
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  plane-names: none array,
  show-axis: bool,
  show-foci: bool,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Cutting plane: three points (A, B, C), or four coefficients a, b, c, d of the equation ax + by + cz + d = 0, or a single argument (dictionary (a:, b:, c:, d:), array of 4 numbers, or array of 3 points)

radius `int` or `float`

Cylinder radius

Default: 1

height `int` or `float`

Cylinder height (distance between the two bases)

Default: 2

base `array`

Center of the “lower” base

Default: `(0, 0, 0)`

axis `array`

Axis direction, from the base toward the upper face (need not be a unit vector)

Default: `(0, 1, 0)`

plane-names `none` or `array`

Names of the three points defining the plane, shown in red (ignored if the plane is specified by an equation)

Default: `none`

show-axis `bool`

Draws the axis with a dash-dot line

Default: `false`

show-foci `bool`

Marks the foci of the cross-section when they exist (ellipse, parabola, hyperbola)

Default: `false`

show-radius `bool`

Draws a radius, with its label, on the visible base

Default: `false`

radius-label `auto` or `none` or `content`

Radius label; `none` omits it, auto give the value

Default: `$r$`

radius-stroke `stroke`

Radius stroke

Default: `black + 0.8pt`

show-centers `bool`

Marks the base centers and the cross-section center when unambiguous

Default: `false`

show-dims `bool`

show the section dimensions with their labels : circle radius, half-axis a and b for a full ellipse (and c, center to focal distance, with show-foci)

Default: `false`

dims-labels `auto` or `dictionary`

labels for the section dimensions : dictionary (r:, a:, b:, c:) whose values replace (r', a, b, c) (auto gives true value), or auto to replace them all

Default: `(:)`

point-labels `dictionary`

Labels placed at anchors (key = anchor name, value = content), for example (c0: O, s: S) ; they are placed outside the solid

Default: `(:)`

label-unit `none` or `str`

Unit added to lengths in auto mode, “cm” for example

Default: `none`

label-digits `int`

Number of digits in auto mode

Default: `2`

label-decimal `str`

Decimal separator in auto mode : “,” ou “.”

Default: `","`

x `array`

Screen image of the x-axis unit vector (oblique projection)

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (oblique projection)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) : by default (-0.5, −0.5), the z-axis points toward the viewer

Default: `(-0.5, -0.5)`

eps `float`

Numerical tolerance (distances)

Default: `1e-6`

per-turn `int`

Number of segments used to draw one full circle or ellipse

Default: `144`

more `content`

Additional CeTZ commands drawn on the same canvas (they can use the anchors)

Default: `()`

section-border `color`

Cross-section outline color

Default: `blue`

section-fill `color`

Cross-section fill (usually semi-transparent)

Default: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Hidden-line style: the value of `dash` for a Typst stroke (“dashed”, “dotted”, …)

Default: `"dashed"`

length `length`

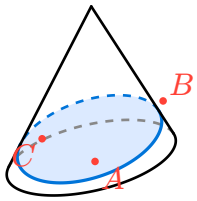
Unit length

Default: `1cm`

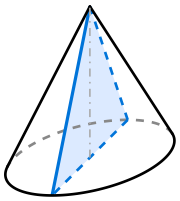
conesection

Draws a cone and its cross-section by a plane in oblique projection (CeTZ). The cone is defined by the radius and center of its base, its height, and its axis direction. Specify the plane as for `polysection`. Depending on the plane, the cross-section is a circle, ellipse, parabola, or part of a hyperbola; a plane passing through the apex gives a triangle. The foci are computed using the Dandelin spheres. Hidden parts are dotted. Anchors: “c0” (base center), “s” (apex), “i0”, “i1”, … (endpoints of the chord on the base), “sc” (center of the cross-section when it is a circle), “f0”, “f1” (foyers).

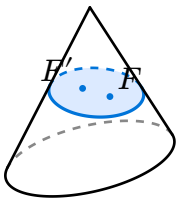
```
// Usage examples:
#conesection((0.3, 0.2, 0.5), (0.6, 0.4,
-0.7), (-0.6, 0.3, 0.1), plane-names:
($A$, $B$, $C$))
```




```
// Plane containing the axis (triangle)
#conesection(1, 0, 0, 0, show-axis:
true)
```



```
// Ellipse with its foci
#conesection(0.3, 1, 0, -0.9, show-foci:
true, point-labels: (f0: $F$, f1: $F'$))
```



Parameters

```
conesection(
  ..plane-args: array dictionary,
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  plane-names: none array,
  show-axis: bool,
  show-foci: bool,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-apex: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Cutting plane: three points (A, B, C), or four coefficients a, b, c, d of the equation ax + by + cz + d = 0, or a single argument (dictionary (a:, b:, c:, d:), array of 4 numbers, or array of 3 points)

radius `int` or `float`

Base radius

Default: 1

height `int` or `float`

Cone height (distance from the base to the apex)

Default: 2

base `array`

Base center

Default: `(0, 0, 0)`

axis `array`

Axis direction, from the base toward the apex (need not be a unit vector)

Default: `(0, 1, 0)`

plane-names `none` or `array`

Names of the three points defining the plane, shown in red (ignored if the plane is specified by an equation)

Default: `none`

show-axis `bool`

Draws the axis with a dash-dot line

Default: `false`

show-foci `bool`

Marks the foci of the cross-section when they exist (ellipse, parabola, hyperbola)

Default: `false`

show-radius `bool`

Draws a radius, with its label, on the base (dotted when hidden)

Default: `false`

radius-label `auto` or `none` or `content`

Radius label; `none` omits it, auto give the value

Default: `$r$`

radius-stroke `stroke`

Radius stroke

Default: `black + 0.8pt`

show-centers `bool`

Marks the base center and the cross-section center when it is a circle

Default: `false`

show-apex `bool`

Marks the apex

Default: `false`

show-dims `bool`

show the section dimensions with their labels : circle radius, half-axis a and b for a full ellipse (and c, center to focal distance, with show-foci)

Default: `false`

dims-labels `auto` or `dictionary`

labels for the section dimensions : dictionary (r:, a:, b:, c:) whose values replace (r', a, b, c) (auto gives true value), or auto to replace them all

Default: `(:)`

point-labels `dictionary`

Labels placed at anchors (key = anchor name, value = content), for example (c0: O, s: S) ; they are placed outside the solid

Default: `(:)`

label-unit `none` or `str`

Unit added to lengths in auto mode, “cm” for example

Default: `none`

label-digits `int`

Number of digits in auto mode

Default: `2`

label-decimal `str`

Decimal separator in auto mode : “,” ou “.”

Default: `","`

x `array`

Screen image of the x-axis unit vector (oblique projection)

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (oblique projection)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) : by default (-0.5, −0.5), the z-axis points toward the viewer

Default: `(-0.5, -0.5)`

eps `float`

Numerical tolerance (distances)

Default: `1e-6`

per-turn `int`

Number of segments used to draw one full circle or ellipse

Default: `144`

more `content`

Additional CeTZ commands drawn on the same canvas (they can use the anchors)

Default: `()`

section-border `color`

Cross-section outline color

Default: `blue`

section-fill `color`

Cross-section fill (usually semi-transparent)

Default: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Hidden-line style: the value of `dash` for a Typst stroke (“dashed”, “dotted”, …)

Default: `"dashed"`

length `length`

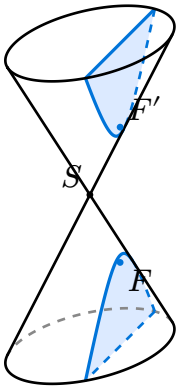
Unit length

Default: `1cm`

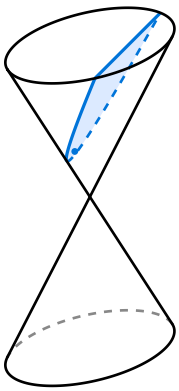
doubleconesection

Draws a double cone (hourglass shape) and its cross-section by a plane in oblique projection (CeTZ). Two cones sharing an apex and axis, extending in opposite directions; each has height `height` and base radius `radius`. Possible cross-sections: circle, ellipse, parabola, hyperbola (two branches, one on each cone), two intersecting lines (plane through the apex), a point, or a line. Anchors: “s” (apex), “c0” (lower base center), “c1” (upper base center), “i0”, “i1”, … (endpoints of the chords on the bases), “sc” (center of a circular section), “f0”, “f1” (foyers). Limitation: the drawing assumes the two cone halves do not occlude each other, which holds as long as the angle between the viewing direction and the axis exceeds the cone half-angle.

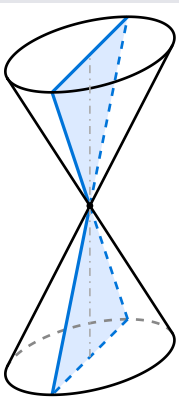
```
// Usage examples:
#doubleconesection(1, 0, 0, -0.4, show-
foci: true, show-apex: true,
  point-labels: (s: $S$, f0: $F$, f1:
  $F'$))
```




```
// Parabola (plane parallel to a
generator)
#doubleconesection(0.894, -0.447, 0,
0.447, show-foci: true)
```



```
// Plane passing through the apex
#doubleconesection(1, 0, 0, 0, show-
axis: true, show-apex: true)
```



Parameters

```
doubleconesection(
  ..plane-args: array dictionary,
  radius: int float,
  height: int float,
  apex: array,
  axis: array,
  plane-names: none array,
  show-axis: bool,
  show-foci: bool,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-apex: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Cutting plane: three points (A, B, C), or four coefficients a, b, c, d of the equation ax + by + cz + d = 0, or a single argument (dictionary (a:, b:, c:, d:), array of 4 numbers, or array of 3 points)

radius `int` or `float`

Base radius of each cone half

Default: `1`

height `int` or `float`

Height of each cone half (distance from the apex to each base)

Default: `2`

apex `array`

Shared apex of the two cones

Default: `(0, 0, 0)`

axis `array`

Axis direction, from the lower cone half to the upper one (need not be a unit vector)

Default: `(0, 1, 0)`

plane-names `none` or `array`

Names of the three points defining the plane, shown in red (ignored if the plane is specified by an equation)

Default: `none`

show-axis `bool`

Draws the axis with a dash-dot line

Default: `false`

show-foci `bool`

Marks the foci of the cross-section when they exist (ellipse, parabola, hyperbola)

Default: `false`

show-radius `bool`

Draws a radius, with its label, on the base facing the viewer

Default: `false`

radius-label `auto` or `none` or `content`

Radius label; `none` omits it, auto give the value

Default: `$r$`

radius-stroke `stroke`

Radius stroke

Default: `black + 0.8pt`

show-centers `bool`

Marks the centers of both bases and the cross-section center when it is a circle

Default: `false`

show-apex `bool`

Marks the apex

Default: `false`

show-dims `bool`

show the section dimensions with their labels : circle radius, half-axis a and b for a full ellipse (and c, center to focal distance, with show-foci)

Default: `false`

dims-labels `auto` or `dictionary`

labels for the section dimensions : dictionary (r:, a:, b:, c:) whose values replace (r', a, b, c) (auto gives true value), or auto to replace them all

Default: `(:)`

point-labels `dictionary`

Labels placed at anchors (key = anchor name, value = content), for example (c0: O, s: S) ; they are placed outside the solid

Default: `(:)`

label-unit `none` or `str`

Unit added to lengths in auto mode, “cm” for example

Default: `none`

label-digits `int`

Number of digits in auto mode

Default: `2`

label-decimal `str`

Decimal separator in auto mode : “,” ou “.”

Default: `","`

x `array`

Screen image of the x-axis unit vector (oblique projection)

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (oblique projection)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) : by default (-0.5, −0.5), the z-axis points toward the viewer

Default: `(-0.5, -0.5)`

eps `float`

Numerical tolerance (distances)

Default: `1e-6`

per-turn `int`

Number of segments used to draw one full circle or ellipse

Default: `144`

more `content`

Additional CeTZ commands drawn on the same canvas (they can use the anchors)

Default: `()`

section-border `color`

Cross-section outline color

Default: `blue`

section-fill `color`

Cross-section fill (usually semi-transparent)

Default: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Hidden-line style: the value of `dash` for a Typst stroke (“dashed”, “dotted”, …)

Default: `"dashed"`

length `length`

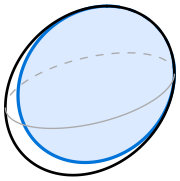
Unit length

Default: `1cm`

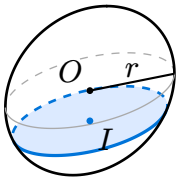
spheresection

Draws a sphere and its cross-section by a plane in oblique projection (CeTZ). The apparent outline is computed exactly (it is an ellipse in oblique projection); an optional equator creates a sense of depth. The cross-section is a circle, with hidden parts shown as dotted lines. Anchors: “o” (center), “sc” (center of the cross-section circle).

```
// Usage examples:
#spheresection(1, 1, 1, -0.5)
```



```
// Horizontal plane with a radius,
centers, and labels
#spheresection(0, 1, 0, -0.6, center:
(0, 1, 0), show-radius: true, show-
centers: true,
  point-labels: (o: $O$, sc: $I$))
```



Parameters

```
spheresection(
  ..plane-args: array dictionary,
  radius: int float,
  center: array,
  axis: array,
  show-equator: bool,
  plane-names: none array,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Cutting plane: three points (A, B, C), or four coefficients a, b, c, d of the equation ax + by + cz + d = 0, or a single argument (dictionary (a:, b:, c:, d:), array of 4 numbers, or array of 3 points)

radius `int` or `float`

Sphere radius

Default: `1`

center `array`

Sphere center

Default: `(0, 0, 0)`

axis `array`

Pole direction (used only to draw the equator)

Default: `(0, 1, 0)`

show-equator `bool`

Draws the equator (a circle perpendicular to the axis), creating a sense of depth

Default: `true`

plane-names `none` or `array`

Names of the three points defining the plane, shown in red (ignored if the plane is specified by an equation)

Default: `none`

show-radius `bool`

Draws a radius on the apparent outline, where it appears at true size

Default: `false`

radius-label `auto` or `none` or `content`

Radius label; `none` omits it, auto give the value

Default: `$r$`

radius-stroke `stroke`

Radius stroke

Default: `black + 0.8pt`

show-centers `bool`

Marks the sphere center and the center of the cross-section circle

Default: `false`

show-dims `bool`

show the section dimensions with their labels : circle radius

Default: `false`

dims-labels `auto` or `dictionary`

labels for the section dimensions : dictionary (r:, a:, b:, c:) whose values replace (r', a, b, c) (auto gives true value), or auto to replace them all

Default: `(:)`

point-labels `dictionary`

Labels placed at anchors (key = anchor name, value = content), for example (c0: O, s: S) ; they are placed outside the solid

Default: `(:)`

label-unit `none` or `str`

Unit added to lengths in auto mode, “cm” for example

Default: `none`

label-digits `int`

Number of digits in auto mode

Default: `2`

label-decimal `str`

Decimal separator in auto mode : “,” ou “.”

Default: `","`

x `array`

Screen image of the x-axis unit vector (oblique projection)

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (oblique projection)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) : by default (-0.5, −0.5), the z-axis points toward the viewer

Default: `(-0.5, -0.5)`

eps `float`

Numerical tolerance (distances)

Default: `1e-6`

per-turn `int`

Number of segments used to draw one full circle or ellipse

Default: `144`

more `content`

Additional CeTZ commands drawn on the same canvas (they can use the anchors)

Default: `()`

section-border `color`

Cross-section outline color

Default: `blue`

section-fill `color`

Cross-section fill (usually semi-transparent)

Default: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Hidden-line style: the value of `dash` for a Typst stroke (“dashed”, “dotted”, …)

Default: `"dashed"`

length `length`

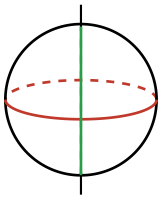
Unit length

Default: `1cm`

globe-view

View parameters (x, y, z) for an orthographic projection centered on the point at longitude `lon` and latitude `lat`. The sphere then appears as a circle, with the south–north axis vertical. Use with the default orientation of `globe` (axis (0, 1, 0), Greenwich (0, 0, 1)). The larger `lat` is, the more the equator opens (depth effect) and the farther the North Pole moves from the top edge of the drawing; between 10 and 20, it appears almost at the highest point.

```
// Usage example:
#globe(..globe-view(lon: 0, lat: 15),
show-axis: true)
```



Parameters

```
globe-view(
  lon: int float,
  lat: int float
) -> dictionary
```

lon `int` or `float`

Longitude of the point at the center of the drawing, in degrees (east is positive)

Default: `20`

lat `int` or `float`

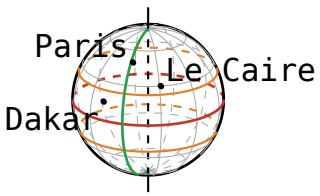
Latitude of the point at the center of the drawing, in degrees (north is positive)

Default: `25`

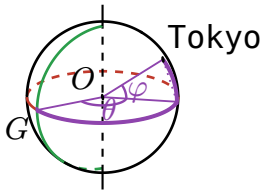
globe

Draws a globe (a variant of `spheresection`) to explain geographic coordinates. Latitude: angle measured from the equator, from −90° (south) to +90° (north). Longitude: angle from the Greenwich meridian, from −180° (west) to +180° (east). All angles are specified in degrees. You can display the equator, the tropics, the Greenwich meridian, parallels and meridians (individually or as a grid), the south–north axis, cities (see `globe-cities`), or points with arbitrary coordinates and, for a point, the construction of its coordinates: longitude angle in the equatorial plane and latitude angle in the plane of its meridian. A plane can also be specified to draw a cross-section (a parallel is a plane perpendicular to the axis, while a meridian is a plane containing it). Oblique projection distorts the sphere into an ellipse; `globe-view` provides an orthographic view. Anchors: “o” (center), “n”, “s” (poles), “g” (point with coordinates (0, 0)), “p0”, “p1”, … and the name of each point, “sc” (cross-section center).

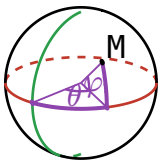
```
// Usage examples:
#globe(..globe-view(lon: 20, lat: 20),
show-axis: true, show-tropics: true,
graticule: 30,
  points: ("Paris", "Le Caire",
  "Dakar"))
```



```
// Constructing a city’s longitude and latitude
#globe(..globe-view(lon: 60, lat: 20), show-axis: true, show-centers: true,
  points: (("city": "Tokyo", angles: true),), point-labels: (o: $O$, g: $G$))
```




```
// Point with arbitrary coordinates
#globe(..globe-view(lon: 40, lat: 20), points: ((name: "M", lat: 35, lon: 60, angles: true),))
```



Parameters

```
globe(
  ..plane-args: array dictionary,
  radius: int float,
  center: array,
  axis: array,
  greenwich: array,
  show-equator: bool,
  show-greenwich: bool,
  show-tropics: bool,
  parallels: array,
  meridians: array,
  graticule: none int float,
  show-axis: bool,
  axis-extension: int float,
  show-centers: bool,
  points: array,
  lon-label: content,
  lat-label: content,
  angle-color: color,
  equator-stroke: stroke,
  greenwich-stroke: stroke,
  tropics-stroke: stroke,
  grid-stroke: stroke,
  axis-stroke: stroke,
  plane-names: none array,
  point-labels: dictionary,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Cutting plane: three points (A, B, C), or four coefficients a, b, c, d of the equation ax + by + cz + d = 0, or a single argument (dictionary (a:, b:, c:, d:), array of 4 numbers, or array of 3 points)

radius `int` or `float`

Sphere radius

Default: 1

center `array`

Sphere center

Default: `(0, 0, 0)`

axis `array`

Direction of the south–north axis, from the South Pole to the North Pole

Default: `(0, 1, 0)`

greenwich `array`

Direction of the point at latitude 0 and longitude 0, projected perpendicular to the axis; with the default values, the Greenwich meridian faces the viewer and east is on the right

Default: `(0, 0, 1)`

show-equator `bool`

Draws the equator

Default: `true`

show-greenwich `bool`

Draws the Greenwich meridian (semicircle from pole to pole)

Default: `true`

show-tropics `bool`

Draws the Tropics of Cancer and Capricorn (latitudes ±23.44°)

Default: `false`

parallels `array`

Additional latitudes to draw, in degrees, for example (30, −45)

Default: `()`

meridians `array`

Additional longitudes to draw, in degrees, for example (90, −120)

Default: `()`

graticule `none` or `int` or `float`

Step, in degrees, for a grid of parallels and meridians (e.g. 30); `none` disables the grid

Default: `none`

show-axis `bool`

Draws the south–north axis (dotted inside and behind the sphere)

Default: `false`

axis-extension `int` or `float`

Length by which the axis extends beyond each pole, as a fraction of the radius

Default: `0.3`

show-centers `bool`

Marks the sphere center and the cross-section center

Default: `false`

points `array`

Points to place: city names from `globe-cities` or dictionaries (name:, lat:, lon:, color: label:, angles:); `angles: true` adds the coordinate construction for the point, `angles: auto` adds the numerical values

Default: `()`

lon-label `content`

Label for the longitude angle

Default: `$theta$`

lat-label `content`

Label for the latitude angle

Default: `$phi$`

angle-color `color`

Color of the angle construction

Default: `rgb("#8e44ad")`

equator-stroke `stroke`

Equator stroke

Default: `rgb("#c0392b") + 1pt`

greenwich-stroke `stroke`

Greenwich meridian stroke

Default: `rgb("#2a9d4a") + 1pt`

tropics-stroke `stroke`

Tropics stroke

Default: `rgb("#e67e22") + 0.8pt`

grid-stroke `stroke`

Grid-parallel and meridian stroke

Default: `gray + 0.4pt`

axis-stroke `stroke`

South–north axis stroke

Default: `black + 0.8pt`

plane-names `none` or `array`

Names of the three points defining the plane, shown in red (ignored if the plane is specified by an equation)

Default: `none`

point-labels `dictionary`

Labels placed at anchors (key = anchor name, value = content), for example (o: O, n: N)

Default: `(:)`

x `array`

Screen image of the x-axis unit vector (oblique projection)

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (oblique projection)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) : by default (-0.5, −0.5), the z-axis points toward the viewer

Default: `(-0.5, -0.5)`

eps `float`

Numerical tolerance (distances)

Default: `1e-6`

per-turn `int`

Number of segments used to draw one full circle or ellipse

Default: `144`

more `content`

Additional CeTZ commands drawn on the same canvas (they can use the anchors)

Default: `()`

section-border `color`

Cross-section outline color

Default: `blue`

section-fill `color`

Cross-section fill (usually semi-transparent)

Default: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Hidden-line style: the value of `dash` for a Typst stroke (“dashed”, “dotted”, …)

Default: `"dashed"`

length `length`

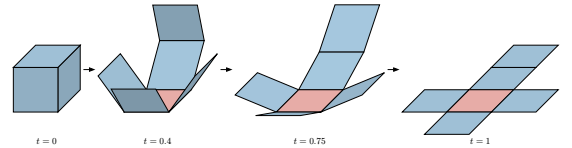
Unit length

Default: `1cm`

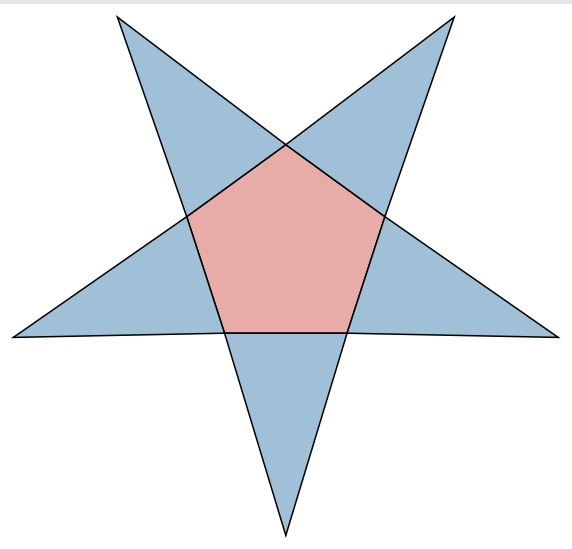
polyunfold

Displays a convex polyhedron, its net, and intermediate unfolding steps (CeTZ). The progress parameter `t` ranges from 0 (closed solid) to 1 (flat net). The requested steps (`steps`) are drawn side by side at the same scale. The solid rests on its bottom face, which remains fixed: the other faces unfold around it. The net is selected from several unfolding trees (breadth-first and depth-first traversal): the first one whose faces do not overlap is selected. Faces are opaque, shaded, and sorted from farthest to nearest; the closed solid is drawn without hidden edges. Anchors, suffixed −1, −2, … when there are multiple steps: “f0”, “f1”, … (face centers) and “v0”, “v1”, … (vertices, in `vertices` order; each vertex is taken from the first face in the tree that contains it).

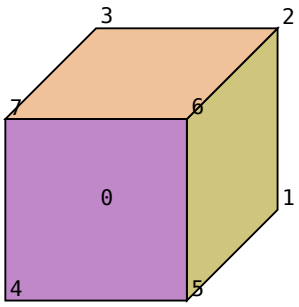
```
// Usage examples:
#polyunfold(steps: (0, 0.4, 0.75, 1),
labels: ($t = 0$, $t = 0.4$, $t = 0.75$,
$t = 1$))
```




```
// Flat net of a pyramid, at true scale
#polyunfold(vertices: regpyramid(5,
side: 2, height: 3), view: "top")
```



```
// One color per face and vertex
numbering
#polyunfold(t: 0, length: 1.2cm,
  palette: (red, green, blue, yellow,
  orange, purple).map(c =>
  c.lighten(60%)),
  more: {
    import cetz.draw: *
    for i in range(8) {
      content("v" + str(i), text(size:
      8pt, str(i)), anchor: "south-
      west", padding: 0.05)
    }
  })
```



Parameters

```
polyunfold(
  vertices: array,
  steps: auto array,
  t: none int float,
  root: auto int,
  up: array,
  view: str,
  columns: auto int,
  gap: int float,
  labels: none array,
  arrows: bool,
  fill: color,
  base-fill: color,
  palette: none array,
  edge-stroke: stroke,
  x: array,
  y: array,
  z: array,
  more: content,
  length: length,
  angle: angle
) -> content
```

vertices `array`

Vertices of the convex polyhedron (defaults to the cube $[-1, 1]^3$), as for `polysection`; the `solidvertices` functions provide them

Default: `(`

```
    (-1, -1, -1), (1, -1, -1), (1, 1, -1), (-1, 1, -1),
    (-1, -1, 1), (1, -1, 1), (1, 1, 1), (-1, 1, 1),
  )
```

steps `auto` or `array`

Values of the unfolding parameter `t` to display side by side, from 0 (closed solid) to 1 (flat net). By default (0, 0.5, 1), or (1,) with `view: "top"`

Default: `auto`

t `none` or `int` or `float`

A single value of `t` (takes precedence over `steps`)

Default: `none`

root `auto` or `int`

Index of the face that remains fixed; `auto`: the bottom face

Default: `auto`

up `array`

“Upward” direction, used to identify the bottom face

Default: `(0, 1, 0)`

view `str`

“oblique”: oblique x, y, z projection (default); “top”: flat net viewed from above, at true scale, oriented like the oblique view

Default: `"oblique"`

columns `auto` or `int`

Number of figures per row; `auto`: all on one row

Default: `auto`

gap `int` or `float`

Spacing between figures (coordinate units)

Default: `0.8`

labels `none` or `array`

Captions displayed below the figures (one per step)

Default: `none`

arrows `bool`

Arrows between consecutive figures on the same row

Default: `true`

fill `color`

Face color

Default: `rgb("#a9cce3")`

base-fill `color`

Color of the fixed face (the base)

Default: `rgb("#f5b7b1")`

palette `none` or `array`

Face colors, repeated as needed; overrides `fill` and `base-fill`

Default: `none`

edge-stroke `stroke`

Edge stroke

Default: `0.7pt + black`

x `array`

Screen image of the x-axis unit vector (“oblique” view) ; no effect with `view: "top"`

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (“oblique” view)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) (“oblique” view)

Default: `(-0.5, -0.5)`

more `content`

Additional CeTZ commands drawn on the same canvas ; the first figure keeps its original coordinates; anchors are available for all figures

Default: `()`

length `length`

Drawing scale: length of one coordinate unit

Default: `1cm`

angle `angle`

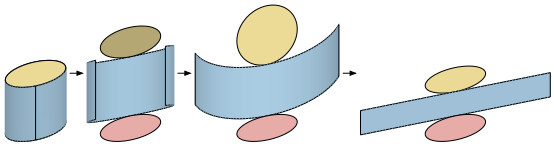
Rotation angle for view:“top”

Default: `0deg`

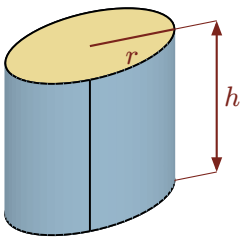
cylunfold

Displays a cylinder, its net, and intermediate unfolding steps (CeTZ). The progress parameter `t` ranges from 0 (closed solid) to 1 (flat net). The base disk remains fixed; the lateral surface unfolds (preserving arc lengths) into a rectangle of length 2πr tangent to the disk, and the second disk tilts with it. Faces are opaque, shaded, and sorted from farthest to nearest. With `cotes: true`, dimensions are added (r, h on the closed solid; 2πr, h, r on the net); labels are placed outside for heights and below the lines for the other dimensions. Anchors, suffixed −1, −2, … when there are multiple steps: “c0” (center of the fixed disk), “c1” (center of the other disk), “pc”, “qc” (contact points, at the bottom and top), “cote-r”, “cote-h”, “cote-p” (midpoints of dimension lines).

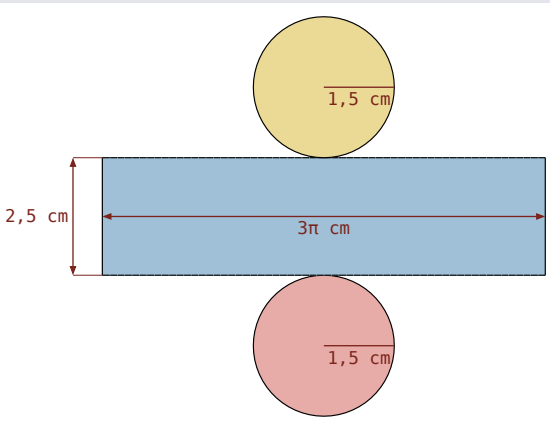
```
// Usage examples:
#cylunfold(steps: (0, 0.3, 0.6, 1))
```



```
// Dimensions on the closed solid
#cylunfold(t: 0, cotes: true)
```



```
// Flat net with actual values
#cylunfold(view: "top", cotes: true,
radius: 1.5, height: 2.5, cote-labels:
auto, cote-unit: "cm")
```



Parameters

```
cylunfold(
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  contact: auto array,
  base-ratio: int float array,
  steps: auto array,
  t: none int float,
  view: str,
  columns: auto int,
  gap: int float,
  labels: none array,
  arrows: bool,
  fill: color,
  base-fill: color,
  top-fill: color,
  segments: int,
  cotes: bool,
  cote-labels: auto dictionary,
  cote-color: color,
  cote-unit: none str,
  cote-digits: int,
  cote-decimal: str,
  edge-stroke: stroke,
  x: array,
  y: array,
  z: array,
  more: content,
  length: length,
  angle: angle
) -> content
```

radius int or float

Cylinder radius

Default: 1

height `int` or `float`

Cylinder height

Default: `2`

base `array`

Center of the base disk, which remains fixed

Default: `(0, 0, 0)`

axis `array`

Axis direction, from the base toward the apex (need not be a unit vector)

Default: `(0, 1, 0)`

contact `auto` or `array`

Direction in the base plane of the point where the solid meets its net; `auto`: the side opposite the observer, the net unfold to the back

Default: `auto`

base-ratio `int` or `float` or `array`

Position(s) of the base disks along the rectangle : ratio between 0 and 1 mesured from the left side of the rectangle in view:“top” ; one number for both disks, or an array (bottom, top) for independant positions

Default: `0.5`

steps `auto` or `array`

Values of the unfolding parameter `t` to display side by side, from 0 (closed solid) to 1 (flat net). By default (0, 0.5, 1), or (1,) with `view: "top"`

Default: `auto`

t `none` or `int` or `float`

A single value of `t` (takes precedence over `steps`)

Default: `none`

view `str`

“oblique”: oblique x, y, z projection (default); “top”: flat net viewed from above, at true scale, oriented like the oblique view

Default: `"oblique"`

columns `auto` or `int`

Number of figures per row; `auto`: all on one row

Default: `auto`

gap `int` or `float`

Spacing between figures (coordinate units)

Default: `0.8`

labels `none` or `array`

Captions displayed below the figures (one per step)

Default: `none`

arrows `bool`

Arrows between consecutive figures on the same row

Default: `true`

fill `color`

Face color

Default: `rgb("#a9cce3")`

base-fill `color`

Color of the fixed disk

Default: `rgb("#f5b7b1")`

top-fill `color`

Color of the second disk

Default: `rgb("#f9e79f")`

segments `int`

Lateral surface mesh resolution (number of facets per revolution)

Default: `60`

cotes `bool`

Adds dimensions: r and h on the closed solid (t = 0, oblique view); 2πr, h, and r on the net (view: “top”)

Default: `false`

cote-labels `auto` or `dictionary`

Dimension labels: dictionary (r:, h:, g:, p:, a:) whose values replace the symbolic labels (the value `auto` uses the actual value), or `auto` to replace all labels

Default: `(:)`

cote-color `color`

Dimension color

Default: `rgb("#7b241c")`

cote-unit `none` or `str`

Unit appended to lengths in automatic mode, for example “cm”

Default: `none`

cote-digits `int`

Number of decimal places in automatic mode

Default: `2`

cote-decimal `str`

Decimal separator in automatic mode: “,” or “.”

Default: `","`

edge-stroke `stroke`

Edge stroke

Default: `0.7pt + black`

x `array`

Screen image of the x-axis unit vector (“oblique” view, like for polysection) ; no effect with `view: "top"`

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (“oblique” view)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) (“oblique” view)

Default: `(-0.5, -0.5)`

more `content`

Additional CeTZ commands drawn on the same canvas ; the first figure keeps its original coordinates; anchors are available for all figures

Default: `()`

length `length`

Drawing scale: length of one coordinate unit

Default: `1cm`

angle `angle`

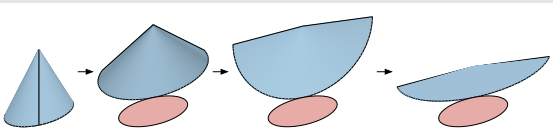
Rotation angle for view:“top”

Default: `0deg`

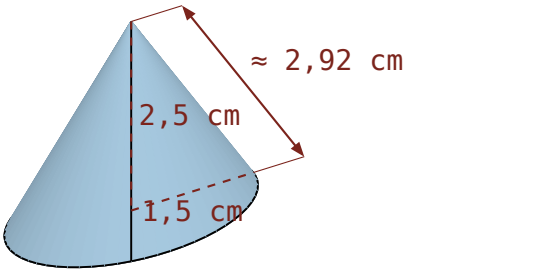
coneunfold

Displays a cone, its net, and intermediate unfolding steps (CeTZ). The progress parameter `t` ranges from 0 (closed solid) to 1 (flat net). The base disk remains fixed; the lateral surface unfolds into a circular sector whose radius equals the slant height, tangent to the disk. Faces are opaque, shaded, and sorted from farthest to nearest. With `cotes: true`, dimensions are added (r, h, g on the closed solid; g, 2πr, the sector angle, and r on the patron) ; the labels are outside for the generator and under for other lines. Anchors, suffixed –1, –2, … when there are multiple steps: “c0” (base center), “s” (apex), “pc” (contact point), “cote-r”, “cote-h”, “cote-g”, “cote-p”, “cote-a” (midpoints of dimension lines).

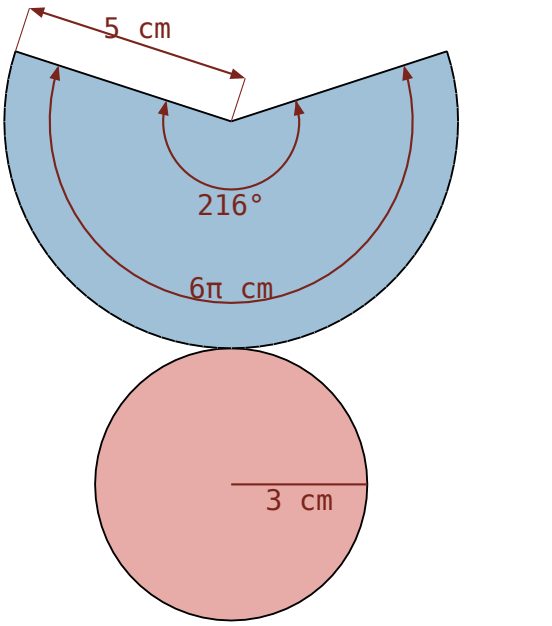
```
// Usage examples:
#coneunfold(steps: (0, 0.3, 0.6, 1))
```



```
// Dimensions on the closed solid, with
actual values
#coneunfold(t: 0, cotes: true, radius:
1.5, height: 2.5, cote-labels: auto,
cote-unit: "cm")
```




```
// Flat net: slant height, arc length
2πr, and sector angle
#coneunfold(view: "top", cotes: true,
radius: 3, height: 4, cote-labels: auto,
cote-unit: "cm", length: 0.6cm)
```



Parameters

```
coneunfold(
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  contact: auto array,
  base-ratio: int float,
  steps: auto array,
  t: none int float,
  view: str,
  columns: auto int,
  gap: int float,
  labels: none array,
  arrows: bool,
  fill: color,
  base-fill: color,
  segments: int,
  cotes: bool,
  cote-labels: auto dictionary,
  cote-color: color,
  cote-unit: none str,
  cote-digits: int,
  cote-decimal: str,
  edge-stroke: stroke,
  x: array,
  y: array,
  z: array,
  more: content,
  length: length,
  angle: angle
) -> content
```

radius `int` or `float`

Cone radius

Default: `1`

height `int` or `float`

Cone height

Default: `2`

base `array`

Base center, fixed

Default: `(0, 0, 0)`

axis `array`

Axis direction, from the base toward the apex (need not be a unit vector)

Default: `(0, 1, 0)`

contact `auto` or `array`

Direction in the base plane of the point where the solid meets its net; `auto`: the side opposite the observer, the net unfold to the back

Default: `auto`

base-ratio `int` or `float`

Position of the base disk along the lateral sector : ratio between 0 and 1 mesured from the top left limit of the arc in view:“top” (0.5 : middle)

Default: `0.5`

steps `auto` or `array`

Values of the unfolding parameter `t` to display side by side, from 0 (closed solid) to 1 (flat net). By default (0, 0.5, 1), or (1,) with `view: "top"`

Default: `auto`

t `none` or `int` or `float`

A single value of `t` (takes precedence over `steps`)

Default: `none`

view `str`

“oblique”: oblique x, y, z projection (default); “top”: flat net viewed from above, at true scale, oriented like the oblique view

Default: `"oblique"`

columns `auto` or `int`

Number of figures per row; `auto`: all on one row

Default: `auto`

gap `int` or `float`

Spacing between figures (coordinate units)

Default: `0.8`

labels `none` or `array`

Captions displayed below the figures (one per step)

Default: `none`

arrows `bool`

Arrows between consecutive figures on the same row

Default: `true`

fill `color`

Face color

Default: `rgb("#a9cce3")`

base-fill `color`

Base color

Default: `rgb("#f5b7b1")`

segments `int`

Lateral surface mesh resolution (number of facets per revolution)

Default: `60`

cotes `bool`

Adds dimensions: r, h, and g on the closed solid (t = 0, oblique view); g, 2πr, and the sector angle (key a) and r on the net (`view: "top"`)

Default: `false`

cote-labels `auto` or `dictionary`

Dimension labels: dictionary (r:, h:, g:, p:, a:) whose values replace the symbolic labels (the value `auto` uses the actual value), or `auto` to replace all labels

Default: `(:)`

cote-color `color`

Dimension color

Default: `rgb("#7b241c")`

cote-unit `none` or `str`

Unit appended to lengths in automatic mode, for example “cm”

Default: `none`

cote-digits `int`

Number of decimal places in automatic mode

Default: `2`

cote-decimal `str`

Decimal separator in automatic mode: “,” or “.”

Default: `","`

edge-stroke `stroke`

Edge stroke

Default: `0.7pt + black`

x `array`

Screen image of the x-axis unit vector (“oblique” view, like for polysection) ; no effect with `view: "top"`

Default: `(1, 0)`

y `array`

Screen image of the y-axis unit vector (“oblique” view)

Default: `(0, 1)`

z `array`

Screen image of the z-axis unit vector (depth direction) (“oblique” view)

Default: `(-0.5, -0.5)`

more `content`

Additional CeTZ commands drawn on the same canvas ; the first figure keeps its original coordinates; anchors are available for all figures

Default: `()`

length `length`

Drawing scale: length of one coordinate unit

Default: `1cm`

angle `angle`

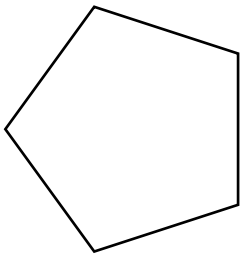
Rotation angle for view:“top”

Default: `0deg`

regpolygon

Vertices of a regular polygon in the plane perpendicular to `axis`, ordered positively around the axis. The polygon is defined by its number of sides and its circumradius or side length. The points are coplanar: they are used to draw the polygon, but not as `vertices` for a solid.

```
// Usage example:
#cetz.canvas({
  import cetz.draw: *
  line(..regpolygon(5, side: 2, axis:
  (0, 0, 1)), close: true)
})
```



Parameters

```
regpolygon(
  n: int,
  radius: none int float,
  side: none int float,
  center: array,
  axis: array,
  phase: int float,
  start: str
) -> array (of points (x, y, z))
```

n `int`

Number of sides (at least 3)

radius `none` or `int` or `float`

Circumradius of the circle (or sphere), i.e. the distance from the center to a vertex; specify either `radius` OR `side`, not both (default `radius: 1`)

Default: `none`

side `none` or `int` or `float`

Side length (edge length for a solid); specify `side` OR `radius`

Default: `none`

center `array`

Polygon center

Default: `(0, 0, 0)`

axis `array`

Normal to the plane containing the polygon (need not be a unit vector)

Default: `(0, 1, 0)`

phase `int` or `float`

Additional rotation around the axis, in degrees, in the positive direction

Default: `0`

start `str`

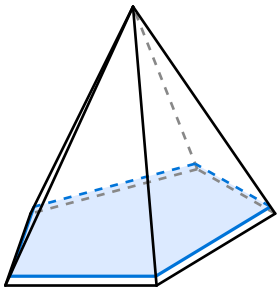
“side”: one side of the base polygon faces the viewer (the +z direction in the oblique view); “vertex”: a vertex

Default: `"side"`

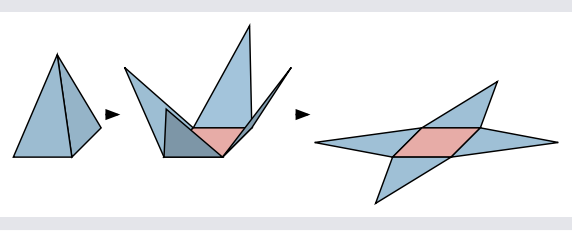
regpyramid

Vertices of a regular pyramid: the n base vertices (ordered positively around the axis), followed by the apex as the last vertex. Use as `vertices:` for `polysection` or `polyunfold`.

```
// Usage examples:
#polysection(0, 1, 0, -0.1, vertices:
regpyramid(5, side: 2, height: 3))
```



```
// Dépliage d'une pyramide à base carrée
#polyunfold(vertices: regpyramid(4, side: 2, height: 3), steps: (0, 0.5, 1), length: 0.5cm)
```



Parameters

```
regpyramid(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str,
  centered: bool
) -> array (of points (x, y, z))
```

n `int`

Number of sides of the base (at least 3)

radius `none` or `int` or `float`

Circumradius of the circle (or sphere), i.e. the distance from the center to a vertex; specify either `radius` OR `side`, not both (default `radius: 1`)

Default: `none`

side `none` or `int` or `float`

Side length (edge length for a solid); specify `side` OR `radius`

Default: `none`

height `int` or `float`

Pyramid height

Default: `1`

center `array`

Base center (or midpoint of the height when `centered: true`)

Default: `(0, 0, 0)`

axis `array`

Axis direction, pointing upward (need not be a unit vector)

Default: `(0, 1, 0)`

phase `int` or `float`

Additional rotation around the axis, in degrees, in the positive direction

Default: `0`

start `str`

“side”: one side of the base polygon faces the viewer (the +z direction in the oblique view); “vertex”: a vertex

Default: `"side"`

centered `bool`

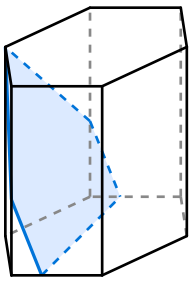
`true`: `center` is the midpoint of the height rather than the base center

Default: `false`

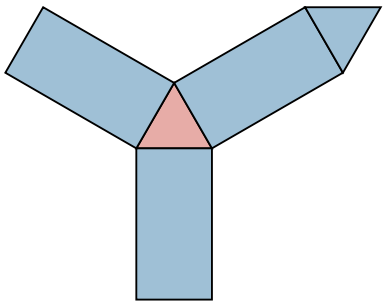
regprism

Vertices of a right prism with a regular base: the n base vertices, followed by the n vertices of the upper face in the same order. Use as `vertices:` for `polysection` or `polyunfold`.

```
// Usage examples:
#polysection(1, 0.4, 0, 0.2, vertices:
regprism(6, radius: 1.2, height: 2.5))
```



```
// Flat net of a triangular prism
#polyunfold(vertices: regprism(3, side:
1, height: 2), view: "top")
```



Parameters

```
regprism(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str,
  centered: bool
) -> array (of points (x, y, z))
```

n `int`

Number of sides of the base (at least 3)

radius `none` or `int` or `float`

Circumradius of the circle (or sphere), i.e. the distance from the center to a vertex; specify either `radius` OR `side`, not both (default `radius: 1`)

Default: `none`

side `none` or `int` or `float`

Side length (edge length for a solid); specify `side` OR `radius`

Default: `none`

height `int` or `float`

Prism height

Default: `1`

center `array`

Base center (or midpoint of the height when `centered: true`)

Default: `(0, 0, 0)`

axis `array`

Axis direction, pointing upward (need not be a unit vector)

Default: `(0, 1, 0)`

phase `int` or `float`

Additional rotation around the axis, in degrees, in the positive direction

Default: `0`

start `str`

“side”: one side of the base polygon faces the viewer (the +z direction in the oblique view); “vertex”: a vertex

Default: `"side"`

centered `bool`

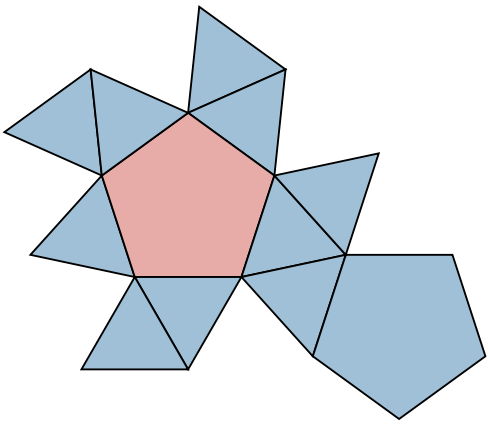
`true`: `center` is the midpoint of the height rather than the base center

Default: `false`

regantiprism

Vertices of an antiprism: as for a prism, but the upper face is rotated by 180°/n relative to the base, producing triangular lateral faces. The same conventions as for `regprism` apply.

```
// Usage example:
#polyunfold(vertices: regantiprism(5,
radius: 1.2, height: 1.2), view: "top")
```



Parameters

```
regantiprism(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str,
  centered: bool
) -> array (of points (x, y, z))
```

n `int`

Number of sides on each base (at least 3)

radius `none` or `int` or `float`

Circumradius of the circle (or sphere), i.e. the distance from the center to a vertex; specify either `radius` OR `side`, not both (default `radius: 1`)

Default: `none`

side `none` or `int` or `float`

Side length (edge length for a solid); specify `side` OR `radius`

Default: `none`

height `int` or `float`

Distance between the planes of the two bases

Default: `1`

center `array`

Base center (or midpoint of the height when `centered: true`)

Default: `(0, 0, 0)`

axis `array`

Axis direction, pointing upward (need not be a unit vector)

Default: `(0, 1, 0)`

phase `int` or `float`

Additional rotation around the axis, in degrees, in the positive direction

Default: `0`

start `str`

“side”: one side of the base polygon faces the viewer (the +z direction in the oblique view); “vertex”: a vertex

Default: `"side"`

centered `bool`

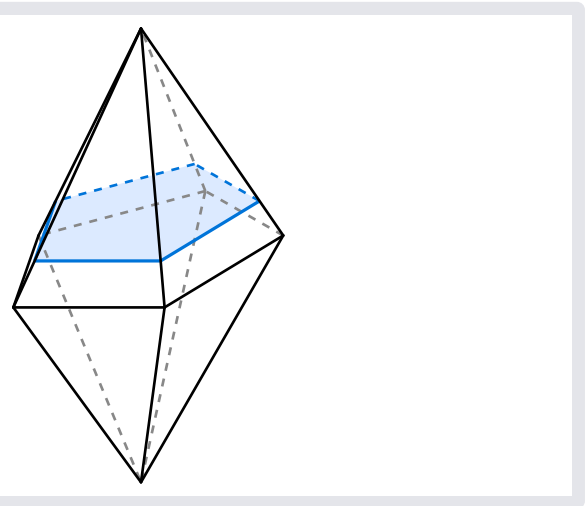
`true`: `center` is the midpoint of the height rather than the base center

Default: `false`

regbipyramid

Vertices of a bipyramid: the n vertices of the middle polygon, followed by the top apex and then the bottom apex.

```
// Usage example:
#polysection(0, 1, 0, -0.5, vertices:
regbipyramid(5, side: 2, height: 3))
```



Parameters

```
regbipyramid(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str
) -> array (of points (x, y, z))
```

n `int`

Number of sides of the middle polygon (at least 3)

radius `none` or `int` or `float`

Circumradius of the circle (or sphere), i.e. the distance from the center to a vertex; specify either `radius` OR `side`, not both (default `radius: 1`)

Default: `none`

side `none` or `int` or `float`

Side length (edge length for a solid); specify `side` OR `radius`

Default: `none`

height `int` or `float`

Height of each pyramid (distance from the middle polygon to each apex)

Default: `1`

center `array`

Center of the middle polygon

Default: `(0, 0, 0)`

axis `array`

Axis direction, pointing upward (need not be a unit vector)

Default: `(0, 1, 0)`

phase `int` or `float`

Additional rotation around the axis, in degrees, in the positive direction

Default: `0`

start `str`

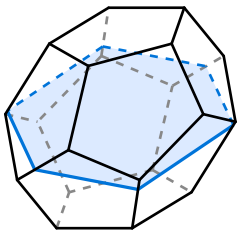
“side”: one side of the base polygon faces the viewer (the +z direction in the oblique view); “vertex”: a vertex

Default: `"side"`

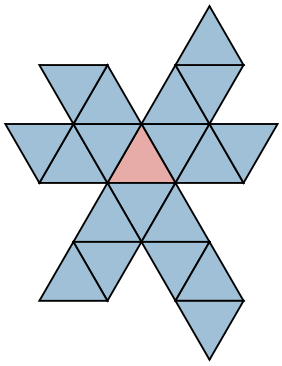
regpolyhedron

Vertices of a regular polyhedron (Platonic solid): “tetrahedron”, “cube”, “octahedron”, “dodecahedron”, or “icosahedron” (English names are accepted). The solid rests on one of its faces, with the axis pointing upward, and one edge of that face facing the viewer; this is also the face that `polyunfold` keeps fixed by default.

```
// Usage examples:
#polysection(0.3, 1, 0.2, 0, vertices:
regpolyhedron("dodécaèdre", side: 1))
```



```
// Flat net of an icosahedron
#polyunfold(vertices:
regpolyhedron("icosaèdre", side: 1.5),
view: "top", length: 0.6cm)
```



Parameters

```
regpolyhedron(
  name: str,
  radius: none int float,
  side: none int float,
  center: array,
  axis: array,
  phase: int float
) -> array (of points (x, y, z))
```

name `str`

Polyhedron name: “tetrahedron”, “cube”, “octahedron”, “dodecahedron”, or “icosahedron”

radius `none` or `int` or `float`

Circumradius of the circle (or sphere), i.e. the distance from the center to a vertex; specify either `radius` OR `side`, not both (default `radius: 1`)

Default: `none`

side `none` or `int` or `float`

Side length (edge length for a solid); specify `side` OR `radius`

Default: `none`

center `array`

Solid center

Default: `(0, 0, 0)`

axis `array`

Axis direction, pointing upward (need not be a unit vector)

Default: `(0, 1, 0)`

phase `int` or `float`

Additional rotation around the axis, in degrees, in the positive direction

Default: `0`